

Category C – Classes and Objects

Java Solutions for GitHub Submission

Based directly on the uploaded Week 6 Category C assignment. The assignment contains five OOP problems (M1–M5).

Source: uploaded assignment, pages 1–5. ■filecite■turn0file0■L5-L14■

M1. Library Inventory Management

Complete Java solution with a separate main class so the solution can be compiled and tested independently.

```
class BookInventory {
    String title;
    String author;
    int copiesAvailable;

    BookInventory(String title, String author, int copiesAvailable) {
        this.title = title;
        this.author = author;
        this.copiesAvailable = copiesAvailable;
    }

    void printEntry() {
        System.out.println(title + " by " + author + " - "
            + copiesAvailable + " copies available");
    }
}

public class M1LibraryInventory {
    public static void main(String[] args) {
        BookInventory[] books = {
            new BookInventory("Clean Code", "Robert C. Martin", 3),
            new BookInventory("Effective Java", "Joshua Bloch", 5),
            new BookInventory("Refactoring", "Martin Fowler", 0),
            new BookInventory("Design Patterns", "GoF", 2)
        };

        for (BookInventory book : books) {
            book.printEntry();
        }
    }
}
```

Expected Output

```
Clean Code by Robert C. Martin - 3 copies available
Effective Java by Joshua Bloch - 5 copies available
Refactoring by Martin Fowler - 0 copies available
Design Patterns by GoF - 2 copies available
```

M2. Payroll Salary Management

Complete Java solution with a separate main class so the solution can be compiled and tested independently.

```
class PayrollAccount {
    private double basicSalary;
    private double bonus;

    public PayrollAccount(double basicSalary) {
        if (basicSalary < 0) {
            System.out.println("Warning: Negative salary not allowed.");
            this.basicSalary = 0;
        } else {
            this.basicSalary = basicSalary;
        }
        this.bonus = 0;
    }

    public void creditBonus(double amount) {
        if (amount <= 0) {
            System.out.println("Bonus rejected: amount must be positive.");
        } else {
            bonus += amount;
            System.out.println("Bonus credited: Rs " + amount);
        }
    }

    public void deductTax(double percent) {
        if (percent < 0 || percent > 100) {
            System.out.println("Tax rejected: percent must be between 0 and 100.");
        } else {
            basicSalary -= basicSalary * percent / 100;
            System.out.println("Tax deducted: " + percent + "%");
        }
    }

    public double getNetSalary() {
        return basicSalary + bonus;
    }
}

public class M2PayrollSalary {
    public static void main(String[] args) {
        PayrollAccount account = new PayrollAccount(50000);

        account.creditBonus(5000);
        account.deductTax(10);

        System.out.println("Net salary: Rs " + account.getNetSalary());
    }
}
```

Expected Output

```
Bonus credited: Rs 5000.0
Tax deducted: 10.0%
Net salary: Rs 50000.0
```

M3. Employee Profile Creation

Complete Java solution with a separate main class so the solution can be compiled and tested independently.

```
class Employee {
    String empId;
    String empName;
    double salary;
    boolean isIntern;

    public Employee(String empId, String empName, double salary) {
        this.empId = empId;
        this.empName = empName;
        this.salary = salary;
        this.isIntern = false;
    }

    public Employee(String empId, String empName) {
        this(empId, empName, 0);
        this.isIntern = true;
    }

    public void printProfile() {
        System.out.println(empId + " | " + empName
            + " | Rs " + salary + " | Intern: " + isIntern);
    }
}

public class M3EmployeeProfile {
    public static void main(String[] args) {
        Employee permanent = new Employee("E-101", "Divya", 65000);
        Employee intern = new Employee("E-102", "Arjun");

        permanent.printProfile();
        intern.printProfile();
    }
}
```

Expected Output

```
E-101 | Divya | Rs 65000.0 | Intern: false
E-102 | Arjun | Rs 0.0 | Intern: true
```

M4. Exam Hall Ticket Reference Management

Complete Java solution with a separate main class so the solution can be compiled and tested independently.

```
class HallTicket {
    String studentName;
    int seatNumber;

    HallTicket(String studentName, int seatNumber) {
        this.studentName = studentName;
        this.seatNumber = seatNumber;
    }
}

public class M4HallTicketReference {
    public static void main(String[] args) {
        HallTicket priya = new HallTicket("Priya", 0);

        // Both variables refer to the same object.
        HallTicket copy = priya;

        // Change through the second reference.
        copy.seatNumber = 45;

        // A separate object with identical field values.
        HallTicket separate = new HallTicket("Priya", 45);

        System.out.println("Priya's seatNumber (via first variable):");
        System.out.println(priya.seatNumber);
        System.out.println("copy == priya: " + (copy == priya));
        System.out.println("separate == priya: " + (separate == priya));
    }
}
```

Expected Output

```
Priya's seatNumber (via first variable):
45
copy == priya: true
separate == priya: false
```

M5. Employee and Company Information Management

Complete Java solution with a separate main class so the solution can be compiled and tested independently.

```
class Employee {
    String empName;
    double salary;

    static String companyName = "Bright Horizon Technologies";
    static int employeeCount = 0;

    Employee(String empName, double salary) {
        this.empName = empName;
        this.salary = salary;
        employeeCount++;
    }

    static void printCompanyInfo() {
        System.out.println(companyName);
        System.out.println("Employees on record: " + employeeCount);
    }
}

public class M5EmployeeCompany {
    public static void main(String[] args) {
        Employee e1 = new Employee("Employee1", 50000);
        Employee e2 = new Employee("Employee2", 55000);
        Employee e3 = new Employee("Employee3", 60000);

        Employee.printCompanyInfo();
    }
}
```

Expected Output

```
Bright Horizon Technologies
Employees on record: 3
```